

LAB 13

Library Management System

What This Lab Brings Together

This is the capstone of the semester. Instead of teaching one new concept, we will build a small but realistic system that uses almost every C++ feature you have learned so far. By the end of this lab, you should be comfortable combining:

- Classes with private data and public functions.
- Constructors with parameter lists.
- Function overloading — two functions with the same name, different signatures.
- Operator overloading — making `+` and `-` work on your own classes.
- Inheritance — building one class on top of another.
- Virtual functions for polymorphism — letting derived classes override behavior.
- Friend classes — letting one class peek into another's private members.
- STL vector for collections of objects.
- Exception handling — `throw` / `try` / `catch` for graceful error recovery.

1. The System We Are Building

We will build a tiny Library Management System with three classes:

Class `Book` (Data Members: `title`, `id`, `quantity`)

- Implement a search function that allows searching by both `id` and `title`, and displays the book if found.
- Use polymorphism mechanisms so that (i) the quantity of any `Book` object can be directly increased or decreased by an integer value, and (ii) two `Book` objects can be directly combined — but only when both share the same `id`, in which case their quantities are added together.
- Provide a `display()` function that prints `title`, `id`, and `quantity`, and can be overridden if needed.

Class `EBook`

Inherits the properties of the `Book` class, adds an extra data member `fileSize` (in MB), and overrides `display()` to also show the file size.

Class Library

Can hold a list of `Book` class objects and, if needed, access the private members of the `Book` class.

Provide the following functionalities:

- `addBook()`, `displayBooks()`
- `borrowBook(id)` — decreases quantity by 1; apply exception handling where needed.
- `returnBook(id)` — increases quantity by 1; apply exception handling where needed.

2. The Setup

The Code

```
#include <iostream>
#include <vector>
#include <stdexcept>
using namespace std;

class Library;           // forward declaration
```

Code Breakdown

Code	What it does
#include <vector>	← Brings in the STL vector — a dynamic array we'll use to store our books inside the Library.
#include <stdexcept>	← Brings in standard exception types like runtime_error that we'll throw when something goes wrong.
class Library;	← Forward declaration. Book uses Library as a friend before Library is defined, so we promise the compiler "Library is a class — full definition is coming later."

Why the forward declaration?

C++ reads top-to-bottom. When the Book class says friend class Library; the compiler has not yet seen Library. The forward declaration tells the compiler "trust me, Library exists" so it can accept the friend statement. The real definition of Library comes after Book.

3. The Book Class

This is the heart of the system. It packs together a constructor, two overloaded search functions, four overloaded operators, a virtual display, and a friend declaration.

The Code

```
class Book {
    string title;
    int    id;
    int    quantity;

public:
    Book(string s, int i, int q) {
        title = s;
        id = i;
        quantity = q;
    }
};
```

```

}

// --- Function overloading: search by id OR by title ---
void search(int x) {
    if (id == x) display();
    else cout << "Oops! Not Found" << endl;
}
void search(string x) {
    if (title == x) display();
    else cout << "Oops! Not Found" << endl;
}

// --- Operator overloading: combine two Books with the same id ---
void operator+(const Book& b) {
    if (id == b.id) quantity += b.quantity;
}
void operator-(const Book& b) {
    if (id == b.id && quantity >= b.quantity)
        quantity -= b.quantity;
}

// --- Operator overloading: adjust quantity by an int ---
void operator+(int x) {
    quantity += x;
}
void operator-(int x) {
    if (quantity >= x) quantity -= x;
}

// --- Virtual display so EBook can override it ---
virtual void display() {
    cout << "Book Title: " << title << endl;
    cout << "Book ID: " << id << endl;
    cout << "Available Copies: " << quantity << endl;
}

friend class Library;          // Library may touch our private members
};

```

Code Breakdown

Code	What it does
title, id, quantity (private)	← All three are private — the outside world cannot touch them directly.
Book(string s, int i, int q)	← Parameterized constructor — every Book MUST be created with these three values.
void search(int x)	← Search by id. The integer parameter tells the compiler this is the id version.

Code	What it does
<code>void search(string x)</code>	← Search by title. Same function name, different parameter type — function overloading.
<code>operator+(const Book& b)</code>	← Combines two books only if their ids match — quantities are summed in place.
<code>operator+(int x)</code>	← Same + symbol, but with an int on the right — directly adds x to quantity. This is operator overloading WITH OVERLOADING — two versions of the + operator.
<code>operator-(const Book& b)</code>	← Mirror of operator+ — removes b.quantity from this book, but only if there are enough copies.
<code>operator-(int x)</code>	← Subtracts x from quantity, guarded so quantity never goes negative.
<code>virtual void display()</code>	← virtual is the key word here — it allows a derived class (EBook) to override this function and have its version called even through a base-class pointer.
<code>friend class Library;</code>	← Grants the entire Library class permission to access Book's private members (id, quantity) directly.



Two kinds of overloading in one class

Notice *search* uses **function overloading** (same name, different parameter types) while *operator+* uses **operator overloading** (same operator, different operand types). The compiler picks the right version in both cases by looking at the types you passed.

4. The EBook Class — Inheritance and Overriding

The Code

```
class EBook : private Book {
    double fileSize;
public:
    EBook(string s, int i, int q, double f) : Book(s, i, q) {
        fileSize = f;
    }

    void display() override {
        Book::display();                // call the base version
        cout << "File Size in (MB): " << fileSize << endl;
    }
};
```

Code Breakdown

Code	What it does
<code>class EBook : private Book</code>	← EBook privately inherits from Book. All inherited members are now PRIVATE inside EBook — they exist but cannot be accessed from outside an EBook object.
<code>double fileSize;</code>	← EBook's own extra data member.
<code>EBook(...) : Book(s, i, q)</code>	← Constructor chaining — pass the first three arguments up to Book's constructor so the inherited members get initialized.
<code>void display() override</code>	← Override Book's virtual display. The override keyword is optional but recommended — the compiler will check that we really are overriding something.
<code>Book::display();</code>	← Explicitly call the base-class version so we get title/id/quantity FIRST, then add the file size on a new line.



Why private inheritance here?

Public inheritance means "EBook IS-A Book" — you can treat an EBook as a Book anywhere. Private inheritance means "EBook is IMPLEMENTED USING a Book" — it reuses Book's machinery but does not advertise itself as a Book to the outside world. That's why our Library (which holds a `vector<Book>`) cannot accept EBook objects directly. The lab uses EBook only for standalone display.

5. The Library Class — Composition, Friendship, and Exceptions

The Code

```

class Library {
    vector<Book> books;

public:
    void addBook(Book b) {
        books.push_back(b);
    }

    void displayBooks() {
        for (int i = 0; i < books.size(); i++) {
            books[i].display();
        }
    }

    void borrowBook(int id) {
        for (int i = 0; i < books.size(); i++) {
            if (id == books[i].id) {                // friend access!
                if (books[i].quantity <= 0) {
                    throw runtime_error("Book not available!");
                }
                books[i] - 1;                        // calls operator-(int)
                cout << "Book Borrowed Successfully!" << endl;
                return;
            }
        }
        throw runtime_error("Invalid Book ID!");
    }

    void returnBook(int id) {
        for (int i = 0; i < books.size(); i++) {
            if (id == books[i].id) {
                books[i] + 1;                        // calls operator+(int)
                cout << "Book Returned Successfully!" << endl;
                return;
            }
        }
        throw runtime_error("Invalid Book ID!");
    }
};

```

Code Breakdown

Code	What it does
<code>vector<Book> books;</code>	← Library holds a dynamic collection of Book objects — uses STL vector so it can grow as more books are added.
<code>addBook(Book b)</code>	← <code>push_back</code> adds the book to the end of the vector.
<code>displayBooks()</code>	← Iterates through every book and calls its <code>display()</code> . Thanks to virtual, if a book were actually an EBook

Code	What it does
	stored as a Book, the EBook version would run — but in this design, the vector holds plain Books.
<code>books[i].id, books[i].quantity</code>	← Direct access to Book's PRIVATE members. This only works because Library was declared as a friend inside Book.
<code>throw runtime_error("...")</code>	← When something goes wrong, throw an exception instead of silently failing. The error message travels with the exception object.
<code>books[i] - 1;</code>	← Uses Book's overloaded operator-(int). Reads naturally: "book minus one copy."
<code>books[i] + 1;</code>	← Uses Book's overloaded operator+(int). Returning a book increases its quantity.



Why friend was needed here

Inside `borrowBook` we access `books[i].id` and `books[i].quantity` directly. Without *friend class Library*; inside Book, this would fail to compile because `id` and `quantity` are private. **friend** is the controlled way to grant one specific outside class privileged access — much better than making everything public.

6. The main() — Putting It All Together

The Code

```
int main() {
    Library lib;
    Book b1("C++", 101, 5);
    Book b2("OOP", 102, 3);
    lib.addBook(b1);
    lib.addBook(b2);

    cout << "\nAll Books:\n";
    lib.displayBooks();

    cout << "\nSearch By ID:\n";
    b1.search(101);           // calls search(int)

    cout << "\nSearch By Title:\n";
    b2.search("OOP");         // calls search(string)
    cout << endl;

    try {
        lib.borrowBook(101);   // valid: succeeds
        lib.returnBook(101);   // valid: succeeds
        lib.borrowBook(999);   // invalid id: throws
        lib.returnBook(555);   // never reached – try block exits
    }
    catch (exception &e) {
        cout << "\nException: " << e.what() << endl;
    }

    cout << "\nEBook Information:\n";
    EBook eb1("AI Basics", 201, 10, 15.5);
    eb1.display();             // overridden version – prints fileSize too

    return 0;
}
```

Tracing the Logic

Code	What it does
Library lib; Book b1(...); Book b2(...);	← Build the library and two books, then add the books to the library.
b1.search(101);	← The argument 101 is an int → compiler picks search(int x).
b2.search("OOP");	← The argument is a string → compiler picks search(string x).

Code	What it does
<code>try { ... } catch (exception &e) { ... }</code>	← Wrap risky operations in try. If anything inside throws, control jumps to the catch block.
<code>lib.borrowBook(101);</code>	← Valid id and quantity > 0 → succeeds, prints success message.
<code>lib.borrowBook(999);</code>	← No book with id 999 → throws <code>runtime_error("Invalid Book ID!")</code> .
<code>lib.returnBook(555);</code>	← Never executes — the previous throw already jumped us into catch.
<code>e.what()</code>	← Returns the message we passed to <code>runtime_error</code> . Prints "Invalid Book ID!"
<code>EBook eb1(...); eb1.display();</code>	← EBook's overridden display runs: first the Book part (<code>Book::display</code>), then the file size line.

Expected Output

► Output

```

All Books:
Book Title: C++
Book ID: 101
Available Copies: 5
Book Title: OOP
Book ID: 102
Available Copies: 3

Search By ID:
Book Title: C++
Book ID: 101
Available Copies: 5

Search By Title:
Book Title: OOP
Book ID: 102
Available Copies: 3

Book Borrowed Successfully!
Book Returned Successfully!

Exception: Invalid Book ID!

EBook Information:
Book Title: AI Basics
Book ID: 201
Available Copies: 10
File Size in (MB): 15.5

```

**Notice how exceptions short-circuit the flow**

When `borrowBook(999)` throws, the very next line — `lib.returnBook(555)` — never runs. Control jumps straight to the catch block. This is a feature: it lets you write the "happy path" linearly and handle ALL the errors in one place at the bottom.

7. Concept Recap — Where Each Idea Lives in the Code

OOP Concept	Where to find it
Function overloading	<code>Book::search(int)</code> and <code>Book::search(string)</code>
Operator overloading	<code>Book::operator+</code> and <code>Book::operator-</code> (with <code>Book</code> and with <code>int</code>)
Inheritance	<code>class EBook : private Book</code>
Virtual functions / override	<code>virtual void display()</code> in <code>Book</code> , overridden in <code>EBook</code>
Constructor chaining	<code>EBook(...) : Book(s, i, q) { ... }</code>
Friend class	<code>friend class Library;</code> inside <code>Book</code>
STL vector	<code>vector<Book> books;</code> inside <code>Library</code>
Exception handling	<code>throw runtime_error(...)</code> in <code>Library</code> , <code>try/catch</code> in <code>main</code>

8. Practice Tasks

Now build two systems of your own using exactly the same design pattern. Each task is graded on the same eight OOP concepts the Library system used — so use the table above as a checklist.

Task 1 — Hospital Management System

Build a small hospital admission system using OOP principles.

Class Patient (data members: name, id, age)

- Implement a search function that allows searching by BOTH id and name and displays the patient if found (function overloading).
- Use operator overloading so that (i) a patient's age can be directly increased by an int (a birthday update), and (ii) two Patient objects can be directly combined — but only when their ids match, in which case their ages average together. Use the `-` operator to decrement age or to separate patients in a similar fashion.
- Provide a virtual `display()` function that prints name, id, and age, and can be overridden if needed.

Class EmergencyPatient

Inherits the properties of the Patient class privately, adds an extra data member `priorityLevel` (int, 1–5 where 1 is most urgent), and overrides `display()` to also show the priority.

Class Hospital

Holds a list of Patient objects and may access Patient's private members. Provide:

- `addPatient()`, `displayPatients()`
- `admitPatient(id)` — marks the patient as admitted; throw an exception if the id does not exist.
- `dischargePatient(id)` — marks the patient as discharged; throw an exception if the id does not exist or the patient was not admitted.

In main()

Add 2 patients, display them, search one by id and one by name. Then inside a try/catch block: admit a valid patient, discharge them, attempt to admit an invalid id (catch the exception). Finally, create one EmergencyPatient object and call `display()` on it.



Hint for Task 1

You'll need an extra bool field like *isAdmitted* inside Patient so the Hospital can track who is currently admitted. Initialize it to *false* in the constructor. Hospital can flip the flag thanks to its *friend* status.

Task 2 — Bank Account Management System

Build a tiny banking system using OOP principles.

Class Account (data members: `holderName`, `accountNumber`, `balance`)

- Implement a search function that allows searching by BOTH `accountNumber` and `holderName` (function overloading).
- Use operator overloading so that (i) the balance can be directly increased or decreased by a double value (a deposit or withdrawal), and (ii) two Account objects can be directly combined with `+` — but only when both belong to the same holder, in which case their balances are added together (treat it as merging two accounts).
- Provide a virtual `display()` function that prints all the details, and can be overridden if needed.

Class SavingsAccount

Inherits the properties of the Account class privately, adds an extra data member `interestRate` (double, in percent), and overrides `display()` to also show the interest rate.

Class Bank

Holds a list of Account objects and may access Account's private members. Provide:

- `addAccount()`, `displayAccounts()`
- `deposit(accNum, amount)` — increases balance; throw an exception if the account does not exist.

- *withdraw(accNum, amount)* — decreases balance; throw an exception if the account does not exist or if balance is insufficient.

In main()

Add 2 accounts, display them, search one by accountNumber and one by holderName. Then inside a try/catch block: deposit into a valid account, withdraw from it successfully, then attempt to withdraw an amount larger than the balance (catch the exception). Finally, create one SavingsAccount object and call display() on it.

Hint for Task 2

Use *double* for balance and amount (not int) since money has decimals. For the insufficient-balance check inside withdraw, throw something like *runtime_error("Insufficient balance!")* — and use a separate message for the invalid-account case so your catch block can give helpful feedback.

Grading checklist (use this for both tasks)

All three classes exist and have correct data members.

Search function is overloaded for two parameter types.

+ and – operators are overloaded BOTH with the class type and with a primitive (int or double).

The derived class uses private inheritance and overrides display().

The container class holds a vector and is declared a friend of the data class.

Exception handling uses throw + try / catch and produces helpful messages.

main() exercises every feature at least once.

Prepared by

Alifa Shanzidah Manarat

Lecturer

Dept. of CSE, BAUST